ASSIGNMENT 5

Database Creation and Table Implementatio

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Create the database for the proposed Airline Reservation System
- Implement all required tables using SQL DDL commands, based on the finalized design
- Define the tables and their relationships (primary keys, foreign keys) correctly

1. Introduction

This assignment implements the finalized 3NF design from Assignment 3 as a physical database. The DDL (Data Definition Language) statements **CREATE DATABASE**, **CREATE TABLE** and **CREATE INDEX** are used. The tables are created in **dependency order**: a table that is referenced by a foreign key must exist before the table that references it.

> **Environment note.** All statements in this report were executed successfully on **SQLite 3.45** with `PRAGMA foreign_keys = ON`, and the outputs shown are the real results. The DDL uses standard SQL types and constraints, so it also runs on MySQL 8 / MariaDB and PostgreSQL with the small dialect changes listed in Section 6.

2. Database Design Summary

The system consists of eight tables. Master data (airline, airport, aircraft, passenger) is stored once, and transaction data (flight, booking, ticket, payment) references it through foreign keys.

Table	Purpose	Primary key	Parent tables (FKs)
airline	Airlines operating flights	airline_id	–
airport	Airports (IATA code, city, country)	airport_code	–
aircraft	Physical aircraft owned by an airline	aircraft_id	airline
flight	A scheduled flight on a given date and route	flight_id	aircraft, airport (origin), airport (destination)
passenger	Registered travellers	passenger_id	–
booking	A reservation made by a passenger (groups tickets)	booking_id	passenger
ticket	One seat of one passenger on one flight	ticket_id	booking, flight, passenger
payment	Payment / refund transactions of a booking	payment_id	booking

Relationships: one airline operates many aircraft; one aircraft is used on many flights; one airport is the origin (and, separately, the destination) of many flights; one flight carries many tickets; one passenger holds many tickets and makes many bookings; one booking contains many tickets and has many payments.

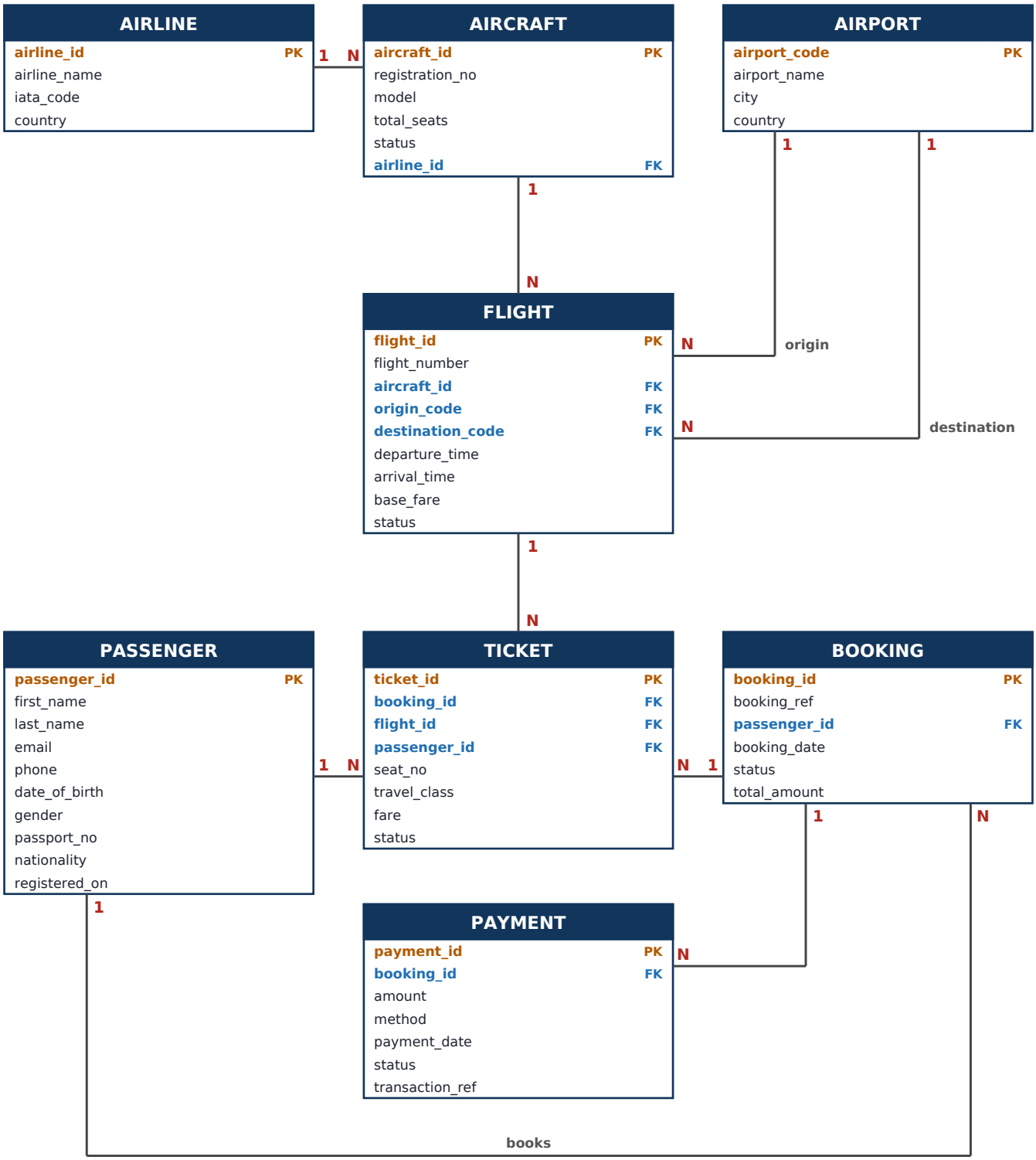


Figure 1: Entity-relationship / table diagram of the Airline Reservation System (PK = primary key, FK = foreign key).

3. Creating the Database

The first step is to create the database (schema) that will contain the tables. In MySQL / MariaDB this is done as shown below. In SQLite the database is a single file (*airline_reservation.db*), which is created automatically on the first connection, so only the foreign-key switch is needed.

Database creation

```
-- MySQL / MariaDB
CREATE DATABASE airline_reservation
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE airline_reservation;

-- SQLite (used for the outputs in this report)
-- sqlite3 airline_reservation.db
PRAGMA foreign_keys = ON;      -- enforce foreign keys for this connection
```

4. Table Creation (DDL)

Each statement below is shown exactly as executed. Column types were chosen as follows: **INTEGER** for surrogate keys and counts, **VARCHAR(n)** for variable text, **CHAR(n)** for fixed-length codes (IATA codes, booking reference), **DECIMAL(10,2)** for money (exact, never floating-point), **DATE** and **DATETIME** for temporal values.

4.1 airline

Parent table: no foreign keys. iata_code and airline_name are UNIQUE.

CREATE TABLE airline

```
CREATE TABLE airline (
    airline_id    INTEGER      PRIMARY KEY,
    airline_name  VARCHAR(80)  NOT NULL UNIQUE,
    iata_code     CHAR(2)      NOT NULL UNIQUE,
    country       VARCHAR(50)  NOT NULL
);
```

4.2 airport

Parent table. The 3-letter IATA airport code is the natural primary key because it is short, stable and already unique worldwide.

CREATE TABLE airport

```
CREATE TABLE airport (
    airport_code  CHAR(3)      PRIMARY KEY,
    airport_name  VARCHAR(100) NOT NULL,
    city          VARCHAR(50)  NOT NULL,
    country       VARCHAR(50)  NOT NULL,
    CONSTRAINT chk_airport_code CHECK (LENGTH(airport_code) = 3)
);
```

4.3 aircraft

References airline. total_seats and status are validated by CHECK constraints.

```
CREATE TABLE aircraft
CREATE TABLE aircraft (
    aircraft_id      INTEGER      PRIMARY KEY,
    registration_no  VARCHAR(10)  NOT NULL UNIQUE,
    model            VARCHAR(40)  NOT NULL,
    total_seats      INTEGER      NOT NULL,
    status           VARCHAR(12)  NOT NULL DEFAULT 'Active',
    airline_id       INTEGER      NOT NULL,
    CONSTRAINT chk_aircraft_seats  CHECK (total_seats BETWEEN 1 AND 900),
    CONSTRAINT chk_aircraft_status CHECK (status IN ('Active','Maintenance','Retired')),
    CONSTRAINT fk_aircraft_airline FOREIGN KEY (airline_id)
        REFERENCES airline (airline_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

4.4 flight

References aircraft and airport twice (origin and destination). A flight number can repeat on different dates, so uniqueness is on (flight_number, departure_time).

```
CREATE TABLE flight
CREATE TABLE flight (
    flight_id         INTEGER       PRIMARY KEY,
    flight_number     VARCHAR(8)    NOT NULL,
    aircraft_id       INTEGER       NOT NULL,
    origin_code       CHAR(3)       NOT NULL,
    destination_code  CHAR(3)       NOT NULL,
    departure_time    DATETIME      NOT NULL,
    arrival_time      DATETIME      NOT NULL,
    base_fare         DECIMAL(10,2) NOT NULL,
    status            VARCHAR(12)   NOT NULL DEFAULT 'Scheduled',
    CONSTRAINT uq_flight_schedule UNIQUE (flight_number, departure_time),
    CONSTRAINT chk_flight_route   CHECK (origin_code <> destination_code),
    CONSTRAINT chk_flight_times   CHECK (arrival_time > departure_time),
    CONSTRAINT chk_flight_fare    CHECK (base_fare > 0),
    CONSTRAINT chk_flight_status  CHECK (status IN ('Scheduled','Delayed','Cancelled','Completed')),
    CONSTRAINT fk_flight_aircraft FOREIGN KEY (aircraft_id)
        REFERENCES aircraft (aircraft_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_flight_origin FOREIGN KEY (origin_code)
        REFERENCES airport (airport_code)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_flight_destination FOREIGN KEY (destination_code)
        REFERENCES airport (airport_code)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

4.5 passenger

Parent table. passport_no and email are unique identifiers; nationality and registered_on have defaults.

```
CREATE TABLE passenger
CREATE TABLE passenger (
    passenger_id   INTEGER      PRIMARY KEY,
    first_name     VARCHAR(40)  NOT NULL,
    last_name      VARCHAR(40)  NOT NULL,
    email          VARCHAR(100) NOT NULL UNIQUE,
    phone          VARCHAR(20)  NOT NULL,
    date_of_birth  DATE         NOT NULL,
    gender         CHAR(1)      NOT NULL,
    passport_no    VARCHAR(12)  NOT NULL UNIQUE,
    nationality    VARCHAR(30)  NOT NULL DEFAULT 'Indian',
    registered_on  DATE         NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT chk_passenger_gender CHECK (gender IN ('M','F','O'))
);
```

4.6 booking

References passenger (the person who made the booking). total_amount defaults to 0 and is later derived from the tickets.

```
CREATE TABLE booking
CREATE TABLE booking (
    booking_id    INTEGER       PRIMARY KEY,
    booking_ref   CHAR(6)       NOT NULL UNIQUE,
    passenger_id  INTEGER       NOT NULL,
    booking_date  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status        VARCHAR(12)   NOT NULL DEFAULT 'Pending',
    total_amount  DECIMAL(10,2) NOT NULL DEFAULT 0,
    CONSTRAINT chk_booking_status CHECK (status IN ('Pending','Confirmed','Cancelled')),
    CONSTRAINT chk_booking_amount CHECK (total_amount >= 0),
    CONSTRAINT fk_booking_passenger FOREIGN KEY (passenger_id)
        REFERENCES passenger (passenger_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

4.7 ticket

The associative (transaction) table: one row per passenger per flight. UNIQUE (flight_id, seat_no) prevents double-booking a seat.

```
CREATE TABLE ticket
CREATE TABLE ticket (
    ticket_id     INTEGER       PRIMARY KEY,
    booking_id    INTEGER       NOT NULL,
    flight_id     INTEGER       NOT NULL,
    passenger_id  INTEGER       NOT NULL,
    seat_no       VARCHAR(4),
    travel_class  VARCHAR(16)   NOT NULL DEFAULT 'Economy',
    fare          DECIMAL(10,2) NOT NULL,
    status        VARCHAR(12)   NOT NULL DEFAULT 'Booked',
    CONSTRAINT uq_ticket_seat     UNIQUE (flight_id, seat_no),
    CONSTRAINT chk_ticket_class   CHECK (travel_class IN ('Economy','Premium Economy','Business')),
    CONSTRAINT chk_ticket_fare    CHECK (fare > 0),
    CONSTRAINT chk_ticket_status  CHECK (status IN ('Booked','Checked-In','Flown','Cancelled')),
    CONSTRAINT chk_ticket_seat    CHECK (status = 'Cancelled' OR seat_no IS NOT NULL),
    CONSTRAINT fk_ticket_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_ticket_flight FOREIGN KEY (flight_id)
        REFERENCES flight (flight_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ticket_passenger FOREIGN KEY (passenger_id)
        REFERENCES passenger (passenger_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

4.8 payment

References booking. A booking may have several payment rows (failed attempts, success, refund).

```
CREATE TABLE payment
CREATE TABLE payment (
    payment_id       INTEGER       PRIMARY KEY,
    booking_id       INTEGER       NOT NULL,
    amount           DECIMAL(10,2) NOT NULL,
    method           VARCHAR(15)   NOT NULL,
    payment_date     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status           VARCHAR(10)   NOT NULL DEFAULT 'Pending',
    transaction_ref  VARCHAR(16)   NOT NULL UNIQUE,
    CONSTRAINT chk_payment_amount CHECK (amount > 0),
    CONSTRAINT chk_payment_method CHECK (method IN ('Credit Card','Debit Card','UPI','Net Banking','Wallet')),
    CONSTRAINT chk_payment_status CHECK (status IN ('Pending','Success','Failed','Refunded')),
    CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

4.9 Indexes

Primary and unique keys are indexed automatically. Additional indexes are created on foreign-key columns and on the columns used by the flight-search query (route and date), because joins and searches are the most frequent operations.

```
CREATE INDEX
CREATE INDEX idx_flight_search     ON flight (origin_code, destination_code, departure_time);
CREATE INDEX idx_flight_aircraft   ON flight (aircraft_id);
CREATE INDEX idx_aircraft_airline  ON aircraft (airline_id);
CREATE INDEX idx_booking_passenger ON booking (passenger_id);
CREATE INDEX idx_ticket_booking    ON ticket (booking_id);
CREATE INDEX idx_ticket_passenger  ON ticket (passenger_id);
CREATE INDEX idx_payment_booking   ON payment (booking_id);
```

5. Verification of the Created Structure

After running the script, the structure was checked with catalog queries. (MySQL equivalents: `SHOW TABLES;` and `DESCRIBE table_name;`)

5.1 List of tables and indexes

```
SHOW TABLES (SQLite catalog query)
SELECT name AS table_name, type FROM sqlite_master WHERE type = 'table' ORDER BY name;
```

```
OUTPUT
+------------+-------+
| table_name | type  |
+------------+-------+
| aircraft   | table |
| airline    | table |
| airport    | table |
| booking    | table |
| flight     | table |
| passenger  | table |
| payment    | table |
| ticket     | table |
+------------+-------+
8 rows in set
```

Indexes created

```
SELECT name AS index_name, tbl_name AS on_table FROM sqlite_master WHERE type = 'index' AND name LIKE 'idx_%' ORDER BY tbl_name, name;
```

OUTPUT

```
+-----------------------+----------+
| index_name            | on_table |
+-----------------------+----------+
| idx_aircraft_airline  | aircraft |
| idx_booking_passenger | booking  |
| idx_flight_aircraft   | flight   |
| idx_flight_search     | flight   |
| idx_payment_booking   | payment  |
| idx_ticket_booking    | ticket   |
| idx_ticket_passenger  | ticket   |
+-----------------------+----------+
7 rows in set
```

5.2 Structure of each table (DESCRIBE)

Columns, data types, nullability, defaults and key membership for every table:

DESCRIBE airline;

OUTPUT

```
+--------------+-------------+------+-----+---------+
| Field        | Type        | Null | Key | Default |
+--------------+-------------+------+-----+---------+
| airline_id   | INTEGER     | NO   | PRI | NULL    |
| airline_name | VARCHAR(80) | NO   | UNI | NULL    |
| iata_code    | CHAR(2)     | NO   | UNI | NULL    |
| country      | VARCHAR(50) | NO   |     | NULL    |
+--------------+-------------+------+-----+---------+
```

DESCRIBE airport;

OUTPUT

```
+--------------+--------------+------+-----+---------+
| Field        | Type         | Null | Key | Default |
+--------------+--------------+------+-----+---------+
| airport_code | CHAR(3)      | NO   | PRI | NULL    |
| airport_name | VARCHAR(100) | NO   |     | NULL    |
| city         | VARCHAR(50)  | NO   |     | NULL    |
| country      | VARCHAR(50)  | NO   |     | NULL    |
+--------------+--------------+------+-----+---------+
```

DESCRIBE aircraft;

OUTPUT

```
+-----------------+-------------+------+----------+----------+
| Field           | Type        | Null | Key      | Default  |
+-----------------+-------------+------+----------+----------+
| aircraft_id     | INTEGER     | NO   | PRI      | NULL     |
| registration_no | VARCHAR(10) | NO   | UNI      | NULL     |
| model           | VARCHAR(40) | NO   |          | NULL     |
| total_seats     | INTEGER     | NO   |          | NULL     |
| status          | VARCHAR(12) | NO   |          | 'Active' |
| airline_id      | INTEGER     | NO   | MUL (FK) | NULL     |
+-----------------+-------------+------+----------+----------+
```

DESCRIBE flight;

```
OUTPUT
+------------------+---------------+------+----------+-------------+
| Field            | Type          | Null | Key      | Default     |
+------------------+---------------+------+----------+-------------+
| flight_id        | INTEGER       | NO   | PRI      | NULL        |
| flight_number    | VARCHAR(8)    | NO   |          | NULL        |
| aircraft_id      | INTEGER       | NO   | MUL (FK) | NULL        |
| origin_code      | CHAR(3)       | NO   | MUL (FK) | NULL        |
| destination_code | CHAR(3)       | NO   | MUL (FK) | NULL        |
| departure_time   | DATETIME      | NO   |          | NULL        |
| arrival_time     | DATETIME      | NO   |          | NULL        |
| base_fare        | DECIMAL(10,2) | NO   |          | NULL        |
| status           | VARCHAR(12)   | NO   |          | 'Scheduled' |
+------------------+---------------+------+----------+-------------+
```

DESCRIBE passenger;

```
OUTPUT
+---------------+--------------+------+-----+--------------+
| Field         | Type         | Null | Key | Default      |
+---------------+--------------+------+-----+--------------+
| passenger_id  | INTEGER      | NO   | PRI | NULL         |
| first_name    | VARCHAR(40)  | NO   |     | NULL         |
| last_name     | VARCHAR(40)  | NO   |     | NULL         |
| email         | VARCHAR(100) | NO   | UNI | NULL         |
| phone         | VARCHAR(20)  | NO   |     | NULL         |
| date_of_birth | DATE         | NO   |     | NULL         |
| gender        | CHAR(1)      | NO   |     | NULL         |
| passport_no   | VARCHAR(12)  | NO   | UNI | NULL         |
| nationality   | VARCHAR(30)  | NO   |     | 'Indian'     |
| registered_on | DATE         | NO   |     | CURRENT_DATE |
+---------------+--------------+------+-----+--------------+
```

DESCRIBE booking;

```
OUTPUT
+--------------+---------------+------+----------+-------------------+
| Field        | Type          | Null | Key      | Default           |
+--------------+---------------+------+----------+-------------------+
| booking_id   | INTEGER       | NO   | PRI      | NULL              |
| booking_ref  | CHAR(6)       | NO   | UNI      | NULL              |
| passenger_id | INTEGER       | NO   | MUL (FK) | NULL              |
| booking_date | DATETIME      | NO   |          | CURRENT_TIMESTAMP |
| status       | VARCHAR(12)   | NO   |          | 'Pending'         |
| total_amount | DECIMAL(10,2) | NO   |          | 0                 |
+--------------+---------------+------+----------+-------------------+
```

DESCRIBE ticket;

```
OUTPUT
+--------------+---------------+------+----------+-----------+
| Field        | Type          | Null | Key      | Default   |
+--------------+---------------+------+----------+-----------+
| ticket_id    | INTEGER       | NO   | PRI      | NULL      |
| booking_id   | INTEGER       | NO   | MUL (FK) | NULL      |
| flight_id    | INTEGER       | NO   | MUL (FK) | NULL      |
| passenger_id | INTEGER       | NO   | MUL (FK) | NULL      |
| seat_no      | VARCHAR(4)    | YES  |          | NULL      |
| travel_class | VARCHAR(16)   | NO   |          | 'Economy' |
| fare         | DECIMAL(10,2) | NO   |          | NULL      |
| status       | VARCHAR(12)   | NO   |          | 'Booked'  |
+--------------+---------------+------+----------+-----------+
```

DESCRIBE payment;

OUTPUT

```
+-----------------+---------------+------+----------+-------------------+
| Field           | Type          | Null | Key      | Default           |
+-----------------+---------------+------+----------+-------------------+
| payment_id      | INTEGER       | NO   | PRI      | NULL              |
| booking_id      | INTEGER       | NO   | MUL (FK) | NULL              |
| amount          | DECIMAL(10,2) | NO   |          | NULL              |
| method          | VARCHAR(15)   | NO   |          | NULL              |
| payment_date    | DATETIME      | NO   |          | CURRENT_TIMESTAMP |
| status          | VARCHAR(10)   | NO   |          | 'Pending'         |
| transaction_ref | VARCHAR(16)   | NO   | UNI      | NULL              |
+-----------------+---------------+------+----------+-------------------+
```

5.3 Foreign-key relationships

The relationships between the tables, read from the database catalog, together with the referential actions:

PRAGMA foreign_key_list (all tables)

```
+-------------+------------------+-------------------------+-------------------+--------------------+
| child_table | fk_column        | references              | update_rule       | delete_rule        |
+-------------+------------------+-------------------------+-------------------+--------------------+
| aircraft    | airline_id       | airline(airline_id)     | ON UPDATE CASCADE | ON DELETE RESTRICT |
| flight      | destination_code | airport(airport_code)   | ON UPDATE CASCADE | ON DELETE RESTRICT |
| flight      | origin_code      | airport(airport_code)   | ON UPDATE CASCADE | ON DELETE RESTRICT |
| flight      | aircraft_id      | aircraft(aircraft_id)   | ON UPDATE CASCADE | ON DELETE RESTRICT |
| booking     | passenger_id     | passenger(passenger_id) | ON UPDATE CASCADE | ON DELETE RESTRICT |
| ticket      | passenger_id     | passenger(passenger_id) | ON UPDATE CASCADE | ON DELETE RESTRICT |
| ticket      | flight_id        | flight(flight_id)       | ON UPDATE CASCADE | ON DELETE RESTRICT |
| ticket      | booking_id       | booking(booking_id)     | ON UPDATE CASCADE | ON DELETE CASCADE  |
| payment     | booking_id       | booking(booking_id)     | ON UPDATE CASCADE | ON DELETE RESTRICT |
+-------------+------------------+-------------------------+-------------------+--------------------+
9 rows in set
```

Rule	Meaning in this system
ON DELETE RESTRICT	A parent row (airline, aircraft, airport, flight, passenger, booking with payments) cannot be deleted while child rows still use it. Protects history.
ON DELETE CASCADE	Used only for ticket → booking: deleting a booking removes its tickets, which have no meaning without it.
ON UPDATE CASCADE	If a key value (for example an IATA code) is corrected, all referencing rows are updated automatically.

5.4 Tables are empty after creation

SQL

```sql
SELECT 'airline' AS table_name, COUNT(*) AS rows_count FROM airline UNION ALL
SELECT 'airport', COUNT(*) FROM airport UNION ALL SELECT 'aircraft', COUNT(*) FROM aircraft UNION ALL
SELECT 'flight', COUNT(*) FROM flight UNION ALL SELECT 'passenger', COUNT(*) FROM passenger UNION ALL
SELECT 'booking', COUNT(*) FROM booking UNION ALL SELECT 'ticket', COUNT(*) FROM ticket UNION ALL
SELECT 'payment', COUNT(*) FROM payment;
```

OUTPUT

```
+------------+------------+
| table_name | rows_count |
+------------+------------+
| airline    |          0 |
| airport    |          0 |
| aircraft   |          0 |
| flight     |          0 |
| passenger  |          0 |
| booking    |          0 |
| ticket     |          0 |
| payment    |          0 |
+------------+------------+
8 rows in set
```

```
Integrity self-check
PRAGMA foreign_key_check;
PRAGMA integrity_check;
```

```
OUTPUT
foreign_key_check -> 0 violations
integrity_check     -> ok
```

6. Portability Notes (MySQL / PostgreSQL)

Item	SQLite (executed)	MySQL 8 / MariaDB	PostgreSQL
Auto-numbered key	INTEGER PRIMARY KEY (auto-assigned if omitted)	INTEGER AUTO_INCREMENT PRIMARY KEY	INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
Date-time type	DATETIME	DATETIME	TIMESTAMP
Enforce foreign keys	PRAGMA foreign_keys = ON	Always on with InnoDB engine	Always on
Create database	File created on connect	CREATE DATABASE …; USE …;	CREATE DATABASE …; then connect
CHECK constraints	Enforced	Enforced from MySQL 8.0.16	Enforced

7. Conclusion

The Airline Reservation System database and its eight tables were created from the finalized design. Primary keys identify every row, nine foreign-key relationships (including both the origin and destination links from flight to airport) link the tables in the intended parent-child order, and indexes support joins and flight search. The catalog queries confirm that all tables, columns and relationships exist and that the empty database passes the integrity checks. The full script is provided as *schema.sql*. The next assignment discusses and tests the integrity constraints in detail, and Assignment 7 loads sample data.